GMSK Project Phase 02

This directory contains the Phase 02 MATLAB implementation of an end-to-end GMSK (Gaussian Minimum Shift Keying) link. It extends the Phase 01 packet modem with a GSM hilly-terrain channel model, coarse burst detection, complex-sample LMS equalization, aligned GMSK demodulation, and soft-decision Viterbi decoding.

System Overview

The simulation is organized as a complete TX -> channel -> RX chain:

1. **Transmitter:** Reads payload bytes from CSV, pads/trims to the waveform payload size, convolutionally encodes the payload, inserts preamble and training bits, then GMSK-modulates the burst.
2. **Channel:** Passes the complex baseband burst through a GSM HT channel and adds AWGN at the requested SNR.
3. **Receiver:** Detects the burst preamble, extracts each burst, normalizes the complex samples, equalizes the burst using known preamble/training reference samples, demodulates to soft symbol metrics, extracts payload metrics, Viterbi-decodes the payload, and writes recovered bytes to CSV.

For waveform ID 0, one burst contains 624 bits:

```
48 preamble bits
+ 544 encoded payload bits  (34 payload bytes, rate-1/2 FEC)
+ 32 training bits
= 624 burst bits

SPS = 4, so one burst is 624 * 4 = 2496 complex samples.
```

The packet layout is interleaved:

```
[Preamble |
 Payload chunk 1 | Training chunk 1 |
 Payload chunk 2 | Training chunk 2 |
 Payload chunk 3 | Training chunk 3 |
 Payload chunk 4 | Training chunk 4]
```

`Waveform.bitMapMask` marks these regions as:

```
1 = preamble, 2 = payload, 3 = training
```

Main Execution Flow

Run the full simulation from:

```
main_system
```

`main_system.m` performs:

```
tx_data.csv
  -> tx_chain.m
  -> channel_model.m
  -> rx_chain.m
  -> rx_data.csv
  -> compare_payload_bytes.m
```

The default channel SNR is set in `main_system.m`:

```
SNR_dB = 30;
```

Receiver Chain

`rx_chain.m` implements the current Option B receiver. The key design point is that equalization is performed on the complex GMSK samples before symbol demapping, using a reference signal generated by the same TX objects used by the transmitter.

High-level RX flow:

```
rx_full
  |
  |-- coarse path, used only for burst detection
  |     RxDemodulator
  |       -> llr_full / dphi metrics at symbol rate
  |     detect_preamble(llr_full, waveform.prmbSeq)
  |       -> burst start bit indices
  |
  |-- per detected burst
        convert detected bit index to sample index
        extract one burst of 2496 complex samples
        normalize by median(abs(rx_burst))
        generate ideal TX reference s_ref
        choose known preamble/training sample indices
        lms_equalizer_complex(rx_norm, s_ref, trainSampleIdx, mu, eq_len,
decisionDelay)
          -> y_eq
        demod_aligned_burst(y_eq, SPS, waveform.burstBlockLen)
          -> one soft metric per burst bit
        select payload metrics with waveform.bitMapMask == 2
        viterbi_soft_decoder(payloadMetrics, TB_LEN)
          -> decoded payload bits
```

```
        pack decoded bits into bytes
        write rx_data.csv
```

Coarse Detection Path

The full received recording is first passed through `RxDemodulator`:

```
llr_full = rxDemod(rx_full);
```

This produces one real phase-difference metric per symbol. These metrics are not the final payload metrics; they are used only to find burst locations. The known preamble from `Waveform.prmbSeq` is correlated against `llr_full` by `detect_preamble.m`. Each detection returns a symbol-rate bit index and a correlation score.

Because `RxDemodulator` compensates for Gaussian filter group delay, the receiver adds the same group-delay offset when mapping the detected bit index back to complex sample space:

```
burstStartSample = (detections(d).index - 1) * SPS + demodGroupDelay + 1;
```

Reference Generation for Equalization

The receiver builds a partially known burst reference:

```
preamble positions -> known waveform.prmbSeq
training positions -> known waveform.trngSeq
payload positions  -> 0 placeholders
```

`gmsk_modulate_burst_ref.m` then passes this reference burst through the actual `GaussianFilter` and `GmskMapper` objects. This is intentional: the equalizer reference must match the transmitter's Gaussian filtering, phase accumulation, lookup-table quantization, and group-delay trimming. A floating-point re-write of the modulator would introduce a reference mismatch that the LMS filter would try to equalize as if it were channel distortion.

The generated `s_ref` is normalized to unit amplitude. The received burst is also normalized before equalization:

```
rx_norm = rx_burst / median(abs(rx_burst));
```

This keeps the NLMS step size `mu` meaningful across different channel gains.

Complex NLMS Equalization

`lms_equalizer_complex.m` adapts a complex FIR equalizer using only known reference samples. In the current receiver, `trainSampleIdx` is derived from the known preamble/training positions, and early samples that cannot support the tap buffer plus decision delay are removed:

```
trainSampleIdx = preamble_sample_indices(waveform.bitMapMask, SPS);
trainSampleIdx = trainSampleIdx(trainSampleIdx >= eq_len + decisionDelay);
```

The equalizer uses normalized LMS:

```
y(n) = w' * u(n)
e(n) = s_ref(n - decisionDelay) - y(n)
w    = w + (mu / (u' * u + eps)) * u * conj(e)
```

The default receiver settings are:

```
mu = 0.05;
eq_len = 13;
TB_LEN = 32;
decisionDelay = floor((eq_len - 1) / 2);
```

`mse_curve` is plotted for each burst so equalizer convergence can be inspected.

Aligned Demodulation and Decoding

After equalization, the current code calls:

```
symMetrics = demod_aligned_burst(y_eq, SPS, waveform.burstBlockLen);
```

This is deliberate. `y_eq` is already aligned to the nominal burst window. Calling `RxDemodulator` again at this point would apply its group-delay trimming a second time and drop valid symbols. `demod_aligned_burst.m` runs `GmskDemapper` and then samples every `SPS` samples from the aligned burst, producing exactly one soft metric per burst bit.

Payload metrics are selected by the mask:

```
payloadMetrics = symMetrics(waveform.bitMapMask == 2);
```

Those 544 soft metrics are passed to `viterbi_soft_decoder.m`, which reconstructs the 272 information bits. The bits are then packed MSB-first into 34 payload bytes.

File Guide

Top-Level Scripts

File	Purpose
`main_system.m`	Runs the complete TX, channel, RX, and payload comparison flow.
`tx_chain.m`	Reads `tx_data.csv`, builds one payload burst, modulates it, and returns the complex TX signal plus signal power.
`rx_chain.m`	Detects bursts, equalizes each burst, demodulates soft metrics, Viterbi-decodes the payload, and writes `rx_data.csv`.
`sweep_equalizer_params.m`	Sweeps `mu`, `eq_len`, and `decisionDelay` to find receiver settings with low byte/bit errors.

Packet and Waveform

File	Purpose
`Waveform.m`	Defines waveform ID 0: preamble length, payload length, encoded payload length, training length, code rate, and `bitMapMask`. It also creates reproducible preamble and training sequences.
`build_packet.m`	Converts payload bytes to MSB-first bits, convolutionally encodes them, and inserts preamble, payload chunks, and training chunks into one burst.
`extract_payload.m`	Utility for selecting payload regions from a burst using the waveform mask.
`preamble_sample_indices.m`	Converts known preamble/training mask positions into sample indices for equalizer training.

Transmitter Blocks

File	Purpose
`convEncoder.m`	Rate-1/2 convolutional encoder using constraint length 7 and generator polynomials `[171 133]`.
`GaussianFilter.m`	Applies Gaussian pulse shaping with `BT = 0.35`, `SPS = 4`, and span of 8 symbols.
`GmskMapper.m`	Converts shaped frequency pulses into complex GMSK samples using phase accumulation and a LUT.
`gmsk_modulate_burst_ref.m`	Reuses `GaussianFilter` and `GmskMapper` at the receiver to generate a TX-identical complex reference for LMS equalization.

Channel

File	Purpose
`channel_model.m`	Applies MATLAB's `stdchan('gsmHTx12c1', ...)` GSM hilly-terrain fading model, normalizes faded power, and adds complex AWGN.

Receiver Blocks

File	Purpose
`RxDemodulator.m`	Runs `GmskDemapper` and samples at symbol times with group-delay compensation. Used for coarse preamble detection on the full recording.
`GmskDemapper.m`	Converts complex GMSK samples into phase-difference metrics.
`detect_preamble.m`	Performs bipolar correlation against the known preamble and returns all peaks above threshold with non-maximum suppression.
`lms_equalizer_complex.m`	Complex NLMS equalizer trained on known preamble/training samples. Returns equalized samples, final taps, and MSE history.
`demod_aligned_burst.m`	Demaps an already aligned burst without applying another group-delay trim. Used after equalization.
`viterbi_soft_decoder.m`	Custom soft-decision Viterbi decoder for the convolutional code.

CSV and Evaluation Utilities

File	Purpose
`read_axi_stream_csv.m`	Reads AXI-stream style CSV data and returns byte/sample fields used by the scripts.
`write_axi_stream_csv.m`	Writes recovered bytes or signal data in AXI-stream style CSV format.
`compare_payload_bytes.m`	Compares the transmitted payload bytes against the recovered bytes and prints mismatch details.

How to Run

1. Open MATLAB.
2. Set the Current Folder to `gmsk-project-phase02`.
3. Ensure `tx_data.csv` exists in this folder.
4. Run:

```
main_system
```

Expected console stages:

```
[TX] Generated ...
[CHANNEL] Applied fading + AWGN ...
```

```
=== Preamble Detection Results ===
[gmsk_modulate_burst_ref]
[lms_equalizer_complex]
[COMPARE] Payload bytes compared ...
=== SYSTEM COMPLETE ===
```

Equalizer Tuning Notes

Use `sweep_equalizer_params.m` when payload errors remain after detection:

```
results = sweep_equalizer_params('tx_data.csv', 30);
```

Typical interpretation of `mse_curve`:

MSE behavior	Likely cause	Adjustment
Oscillates or diverges	`mu` too large	Reduce `mu`
Barely decreases	`mu` too small	Increase `mu`
Drops, then plateaus high	Equalizer too short for the channel spread	Increase `eq_len`
Drops, then rises	Overfitting or step size too large	Reduce `mu` or `eq_len`
Drops cleanly to a low value	Equalizer is converging	Keep settings

For waveform ID 0, the receiver has 32 training bits and 4 samples per bit, so there are 128 known training samples before any validity trimming. Practical `eq_len` values are usually 9, 13, or 17.

Requirements

This code is written for MATLAB. `channel_model.m` uses Communications Toolbox features such as `stdchan` and `physconst`.